

LAPORAN JOBSHEET
PEMROGRAMAN BERBASIS FRAMEWORK



Nama : Yan Daffa Putra Liandhie

NIM: 2341720142

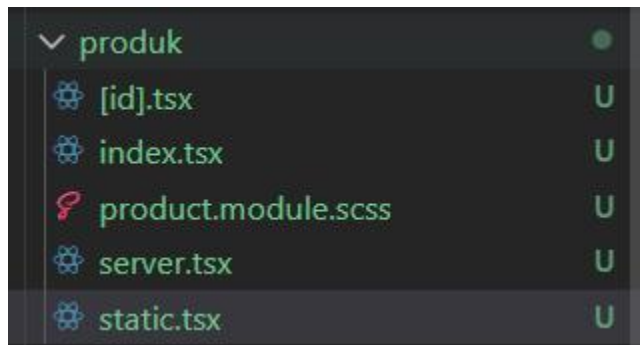
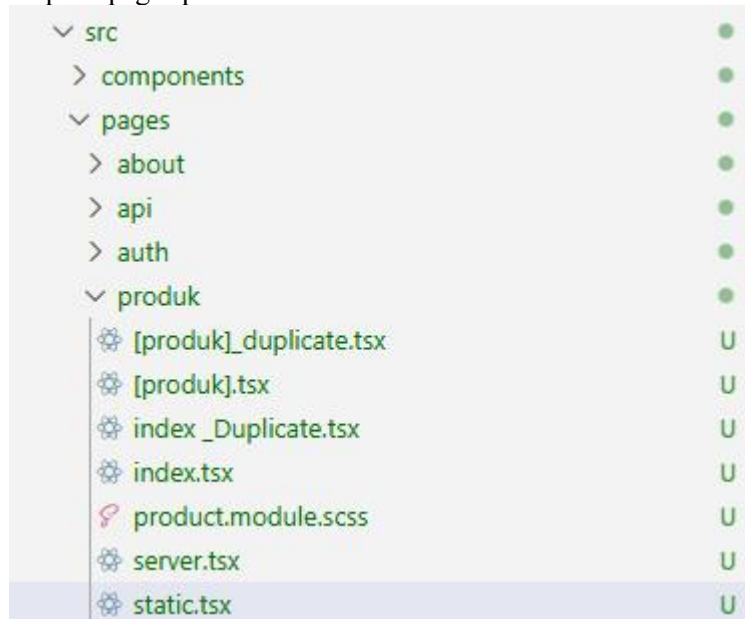
Kelas: TI 3E

POLITEKNIK NEGERI MALANG TAHUN 2026

✂ C. Langkah Praktikum

◆ Bagian 1 – Setup Halaman Static

1. Buat file baru pada pages/products/static.tsx



2. Modifikasi file static.tsx

static.tsx U X

9.praktikum-Static_side_generation > my-app > src > pages > produk > static.tsx > halamanProdukStatic

```
1
2 import TampilanProduk from "../../views/product";
3 import { ProductType } from "../../types/Product.type";
4
5
6
Windsurf: Refactor | Explain | Generate JSDoc | X
7 const halamanProdukStatic = (props:{products:ProductType[]}) => {
8   const { products } = props;
9   return (
10     <div>
11       <h1>Halaman Produk Static</h1>
12       <TampilanProduk products={products} />
13     </div>
14   );
15 };
16
17 export default halamanProdukStatic;
18
Windsurf: Refactor | Explain | Generate JSDoc | X
19 export async function getStaticProps() {
20   const res = await fetch('http://127.0.0.1:3000/api/produk');
21   // const response: ProductType[] = await res.json();
22   const response: { data: ProductType[] } = await res.json();
23
24   // console.log("Data produk yang diambil dari API:", response);
25   return {
26     props: {
27       products: response.data,
28     }
29   }
30 }
31
```

src > pages > produk > static.tsx > getStaticProps

```
1 import TampilanProduk from "../../views/produk";
2 import { ProductType } from "../../types/Product.type";
3
Windsurf: Refactor | Explain | Generate JSDoc | X
4 const halamanProdukStatic = (props:{products:ProductType[]}) => {
5   const {products} = props;
6   return (
7     <div>
8       <h1>Halaman Produk Static</h1>
9       <TampilanProduk products={products} />
10     </div>
11   );
12 };
13 export default halamanProdukStatic;
14
Windsurf: Refactor | Explain | Generate JSDoc | X
15 export async function getStaticProps() {
16   const res = await fetch('http://localhost:3000/api/produk');
17   const response:{data : ProductType[]} = await res.json();
18   return {
19     props: {
20       products: response.data
21     }
22   }
23 }
```

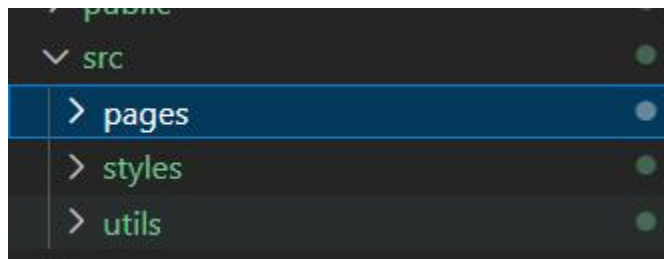
 Catatan:

- Mirip dengan SSR

- Perbedaan hanya pada nama method
-

◆ Bagian 3 – Build Production Mode

1. Pindah beberapa folder diluar pages antara lain
 - Untuk menghindari error maka folder Views, utils, styles dipindah di luar folder src sehingga susunan folder pada src sebagai berikut



2. Jalankan: npm run build
 - Jalankan npm run dev dan pastikan ini jalan (jangan distop saat ngebuild), jadi buka dua terminal
 - o Terminal 1 : jalankan aplikasi npm run dev

```
D:\tugas\semester 6\framework\js10\code\my-app>npm run build
```

```

Route (pages)
├── ○ /
│   ├── /_app
│   └── ○ /404
├── ○ /about
├── f /api/hello
├── f /api/produk
├── ○ /auth/login
├── ○ /auth/register
├── ○ /blog/[slug]
├── ○ /category/[...slug]
├── ○ /components/layouts/Appshell
├── ○ /components/layouts/navbar
├── ○ /produk
├── ○ /produk/[id]
├── f /produk/server
├── ● /produk/static (1935 ms)
├── ○ /profile
├── ○ /profile/edit
├── ○ /setting/app
├── ○ /setting/user
├── ○ /setting/user/password
├── ○ /shop/[...slug]
├── ○ /views/auth/login
├── ○ /views/auth/register
├── ○ /views/produk
├── ○ /views/produk/detail
├── ○ /views/produk/hero_section
├── ○ /views/produk/main_section
├── ○ (Static) prerendered as static content
├── ● (SSG) prerendered as static HTML (uses getStaticProps)
└── f (Dynamic) server-rendered on demand

```

○ Terminal 2 : build aplikasi

```

D:\tugas\semester 6\framework\js10\code\my-app>npm run dev

> my-app@0.1.0 dev
> next dev

▲ Next.js 16.1.6 (Turbopack)
- Local:      http://localhost:3000
- Network:    http://172.29.31.98:3000
- Environments: .env.local

✓ Starting...
✓ Ready in 1387ms
GET /api/produk 200 in 1282ms (compile: 700ms, render: 582ms)

```

- Hal ini dikarenakan aplikasi berjalan di local <http://localhost:3000/api/produk> (jika berhasil maka akan muncul pesan route(pages))

Route (pages)

- o /
 - o /_app
 - o /404
 - o /about
 - f /api/hello
 - f /api/produk
 - o /auth/login
 - o /auth/register
 - o /produk
 - o /produk/[produk]
 - o /produk/[produk]_duplicate
 - o /produk/index_Duplicate
 - f /produk/server
 - /produk/static (1137 ms)
 - o /setting/app
 - o /shop/[[...slug]]
 - o /user
 - o /user/password
-
- o (Static) prerendered as static content
 - (SSG) prerendered as static HTML (uses `getStaticProps`)
 - f (Dynamic) server-rendered on demand

Note : jika error maka lakukan langkah berikut :

- hapus folder .next menggunakan perintah Remove-Item -Recurse -Force .next
- ketik perintah npm run dev

2. Jika berhasil: npm run start

Jika saat menjalankan npm run start muncul error :

```
PS D:\1.MateriKuliah\2026-ganjil\PemogramanFramework\Code\9.praktikum-Static_side_generation\my-app> npm run start

> my-app@0.1.0 start
> next start

✖ Failed to start server
Error: listen EADDRINUSE: address already in use :::3000
    at <unknown> (Error: listen EADDRINUSE: address already in use :::3000)
    at new Promise (<anonymous>) {
  code: 'EADDRINUSE',
  errno: -4091,
  syscall: 'listen',
  address: ':::',
  port: 3000
}
```

Maka ada perintah npm run dev yang sedang jalan , matikan terlebih dahulu setelah lakukan perintah npm run start

```
PS D:\1.MateriKuliah\2026-ganjil\PemogramanFramework\Code\9.praktikum-Static_side_generation\my-app> npm run start

> my-app@0.1.0 start
> next start

▲ Next.js 16.1.6
- Local:      http://localhost:3000
- Network:    http://192.168.1.22:3000

✓ Starting...
✓ Ready in 363ms
```

```
D:\tugas\semester 6\framework\js10\code\my-app>npm run start

> my-app@0.1.0 start
> next start

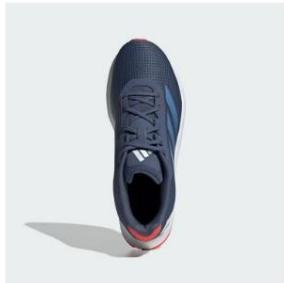
▲ Next.js 16.1.6
- Local:      http://localhost:3000
- Network:    http://192.168.56.1:3000

✓ Starting...
✓ Ready in 848ms
```

3. Akses: <http://localhost:3000/products/static>

Halaman Produk Static

Daftar Produk



Sepatu Duramo SL

Men's Shoes

Rp 900.000,00



SEPATU SAMBA OG

Men's Shoes

Rp 2.000.000,00

Navbar

Halaman Produk Static

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000



sepatu pria

sepatu

Rp 300,000



celana

celana

Rp 150,000



baju polo

pakaian

Rp 100,000

◆ Bagian 4 – Pengujian Perubahan Data Uji

1 – Tambah Data di Database

1. Buka database firebasenya

- Tambahkan produk baru di database.

The screenshot shows the Firebase console interface for adding a new document to a Firestore database. The document is being added to the 'products' collection. The fields being added are:

- name** (string): sepatu kasogi
- image** (string): http
- price** (number): 500000
- category** (string): sepatu lain

Below the form, the 'Database' tab is selected, showing a tree view of the database structure. The 'products' collection is expanded, and the document being added is visible in the list of documents.

Document ID ?

hOErnM6f7z2QtMqAqAUW

Field	Type
name	string
String	
sepatu kasogi	
image	string
String	
http	
price	number
Value	
500000	
category	string
String	
sepatu lain	

+ Add field

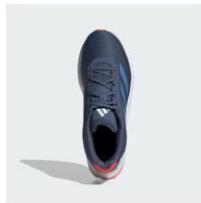
(default)	products	hOErnM6f7z2QtMqAqAUW
+ Start collection	+ Add document	+ Start collection
products >	R6ikqN3cN9bUiU2dRBvG	+ Add field
	VU8Zce7kpPHT5ukib7MS	category: "sepatu lain"
	hOErnM6f7z2QtMqAqAUW >	image: "http"
	rW0jdtP2scnnl1v0YaN4	name: "sepatu kasogi"
	ut70eVpZaWBQvwJ2PSR2	price: 500000
	whG1AfdQjJw1PerAJFGW	

2. Buka halaman:

- /products (CSR) → Data bertambah

Daftar Produk

sepatu kasogi



sepatu kasogi

sepatu lain

Rp 500,000

Sepatu Duramo SL

Men's Shoes

Rp 900.000,00

SEPATU SAMBA OG

Men's Shoes

Rp 2.000.000,00

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000

sepatu kasogi



sepatu kasogi

sepatu lain

Rp 500,000

sepatu pria

sepatu

Rp 300,000



celana

celana

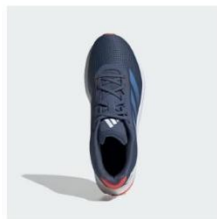
Rp 150,000

- /products/server (SSR) → Data bertambah

Halaman Produk Server

Daftar Produk

sepatu kasogi



sepatu kasogi

sepatu lain

Rp 500,000

Sepatu Duramo SL

Men's Shoes

Rp 900.000,00

SEPATU SAMBA OG

Men's Shoes

Rp 2.000.000,00

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000

sepatu kasogi



sepatu kasogi

sepatu lain

Rp 500,000

sepatu pria

sepatu

Rp 300,000

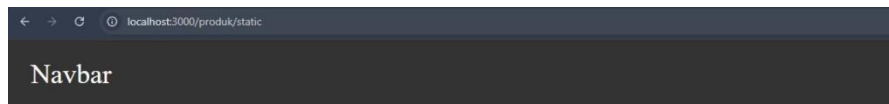


celana

celana

Rp 150,000

- /products/static (SSG) → Data tidak berubah



Halaman Produk Static

Daftar Produk



Daftar Produk



Uji 2 – Build Ulang

1. Jalankan kembali:

- npm run build
 - lakukan secara bersamaan dengan npm run dev saat melakukan npm run build

```

D:\tugas\semester 6\framework\js10\code\my-app>npm run dev

> my-app@0.1.0 dev
> next dev

✓ Console Ninja extension is connected to Next.js, see https://tinyurl.com/2vt8jxzw
⚠ Console Ninja Turbopack support is a PRO feature, currently available to community users while in preview
▲ Next.js 16.1.6 (Turbopack)
  - Local:    http://localhost:3000
  - Network:  http://192.168.56.1:3000
  - Environments: .env.local

✓ Starting...
✓ Ready in 3.5s
          
```

```

Route (pages)
├─ /
├─ /_app
├─ /404
├─ /about
├─ f /api/hello
├─ f /api/produk
├─ /auth/login
├─ /auth/register
├─ /blog/[slug]
├─ /category/[...slug]
├─ /components/layouts/Appshell
├─ /components/layouts/navbar
├─ /produk
├─ /produk/[id]
├─ f /produk/server
├─ ● /produk/static (11323 ms)
├─ /profile
├─ /profile/edit
├─ /setting/app
├─ /setting/user
├─ /setting/user/password
├─ /shop/[...slug]
├─ /views/auth/login
├─ /views/auth/register
├─ /views/produk
├─ /views/produk/detail
├─ /views/produk/hero_section
└─ /views/produk/main_section
          
```

- npm run start
 - npm run dev stop terlebih dahulu setelah itu npm run start

```

D:\tugas\semester 6\framework\js10\code\my-app>npm run start
rt

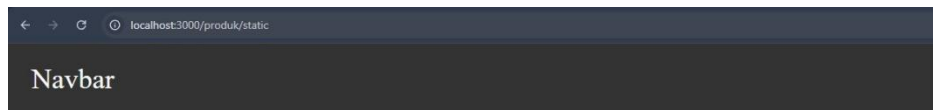
> my-app@0.1.0 start
> next start

▲ Next.js 16.1.6
- Local:      http://localhost:3000
- Network:    http://192.168.56.1:3000

✓ Starting...
✓ Ready in 951ms

```

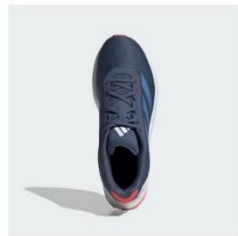
2. Refresh halaman static
→ Data baru muncul



Halaman Produk Static

Daftar Produk

sepatu kasogi



sepatu kasogi

Sepatu Duramo SL

SEPATU SAMBA OG

sepatu lain

Men's Shoes

Men's Shoes

Rp 500,000

Rp 900.000,00

Rp 2.000.000,00



Halaman Produk Static

Daftar Produk



sepatu kasogi



jaket

kemeja

sepatu kasogi

sepatu pria

celana

jaket

pakaian

sepatu lain

sepatu

celana

Rp 200,000

Rp 130,000

Rp 500,000

Rp 300,000

Rp 150,000

D. Tugas Praktikum

Tugas Individu

1. Buat 3 halaman:
 - CSR

```
// export default Kategori;
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import TampilanProduk from "../views/produk";
import useSWR from "swr";
```

```
const fetcher = (url: string) => fetch(url).then((res) => res.json());
```

Windsurf: Refactor | Explain | Generate JSDoc | X

```
const kategori = () => {
  // const [isLogin, setIsLogin] = useState(false);
  // const { push } = useRouter();
  const [products, setProducts] = useState([]);
  // console.log("products:", products);

  // useEffect(() => {
  //   if (!isLogin) {
  //     push("/auth/login");
  //   }
  // }, []);

  const { data, error, isLoading } = useSWR("/api/produk", fetcher);

  // useEffect(() => {
  //   fetch("/api/produk")
  //     .then((response) => response.json())
  //     .then((responseData) => {
  //       setProducts(responseData.data);
  //       // console.log("Data produk:", responseData.data);
  //     })
  //     .catch((error) => {
  //       console.error("Error fetching produk:", error);
  //     });
  // }, []);

  return (
    <div>
      <TampilanProduk products={isLoading ? [] : data.data} />
    </div>
  );
};

export default kategori;
```

Navbar

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000

 sepatu kasogi



sepatu kasogi

sepatu lain

Rp 500,000



sepatu pria

sepatu

Rp 300,000

celana

celana

Rp 150,000

○ SSR

```

1 import TampilanProduk from "../views/produk";
2 import { ProductType } from "../../types/Product.type";
3
4 const halamanProdukServer = (props: { products: ProductType[] }) => {
5   const { products } = props;
6   return (
7     <div>
8       <h1>Halaman Produk Server</h1>
9       <TampilanProduk products={products} />
10    </div>
11  );
12 };
13
14 export default halamanProdukServer;
15
16 // Fungsi getServerSideProps akan dipanggil setiap kali halaman ini diakses,
17 // dan akan mengambil data produk dari API sebelum merender halaman.
18 export async function getServerSideProps() {
19   const res = await fetch("http://localhost:3000/api/produk");
20   const response = await res.json();
21   // console.log("Data produk yang diambil dari API:", response);
22
23   return {
24     props: {
25       products: response.data, // Pastikan untuk memberikan nilai default jika data tidak tersedia
26     },
27   };
28 }

```



halaman Produk Server

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000

 sepatu kasogi



sepatu kasogi

sepatu lain

Rp 500,000



sepatu pria

sepatu

Rp 300,000

celana

celana

Rp 150,000

○ SSG


```

src > pages > produk > static.tsx > getStaticProps > res
1  import TampilanProduk from "../views/produk";
2  import { ProductType } from "../../types/Product.type";
3
Windsurf: Refactor | Explain | Generate JSDoc | X
4  const halamanProdukStatic = (props: {products: ProductType[]}) => {
5      const {products} = props;
6      return (
7          <div>
8              <h1>Halaman Produk Static</h1>
9              <TampilanProduk products={products} />
10             </div>
11         );
12     };
13     export default halamanProdukStatic;
14
Windsurf: Refactor | Explain | Generate JSDoc | X
15     export async function getStaticProps() {
16         const res = await fetch('http://localhost:3000/api/produk');
17         const response: {data : ProductType[]} = await res.json();
18         return {
19             props: {
20                 products: response.data
21             }
22         };
23     }

```



alaman Produk Static

Daftar Produk



2. Lakukan pengujian:

- Tambah data

Document ID [hOErnM6f7z2QtMqAqAUW](#)

Field: = Type:

String:

Field: = Type:

String:

Field: = Type: Value:

Field: = Type:

String:

(default) + Start collection products >	products + Add document R6ikqN3cN9bUiU2dRBvG VU8Zce7kpPHT5ukib7MS hOErnM6f7z2QtMqAqAUW > rW0jdtP2scnnl1v0YaN4 ut70eVpZaWBQvwJ2PSR2 whGLAfdQjjwLPerAJFGW	hOErnM6f7z2QtMqAqAUW + Start collection + Add field category: "sepatu lain" image: "http" name: "sepatu kasogi" price: 500000
--	---	--

/produk (CSR)

Navbar

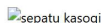
Daftar Produk



jaket
jaket
Rp 200,000



kemeja
pakaian
Rp 130,000



sepatu kasogi
sepatu lain
Rp 500,000



sepatu pria
sepatu
Rp 300,000



celana
celana
Rp 150,000

/produk/server (SSR)

Navbar

Halaman Produk Server

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000

 sepatu kasogi



sepatu kasogi

sepatu lain

Rp 500,000



sepatu pria

sepatu

Rp 300,000

celana

celana

Rp 150,000

/produk/static (SSG)

- sebelum build ulang

localhost:3000/produk/static

Navbar

Halaman Produk Static

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000



sepatu pria

sepatu

Rp 300,000



celana

celana

Rp 150,000



baju polo

pakaian

Rp 100,000

- Sesudah build ulang

localhost:3000/produk/static

Navbar

Halaman Produk Static

Daftar Produk



jaket

jaket

Rp 200,000



kemeja

pakaian

Rp 130,000

 sepatu kasogi



sepatu kasogi

sepatu lain

Rp 500,000



sepatu pria

sepatu

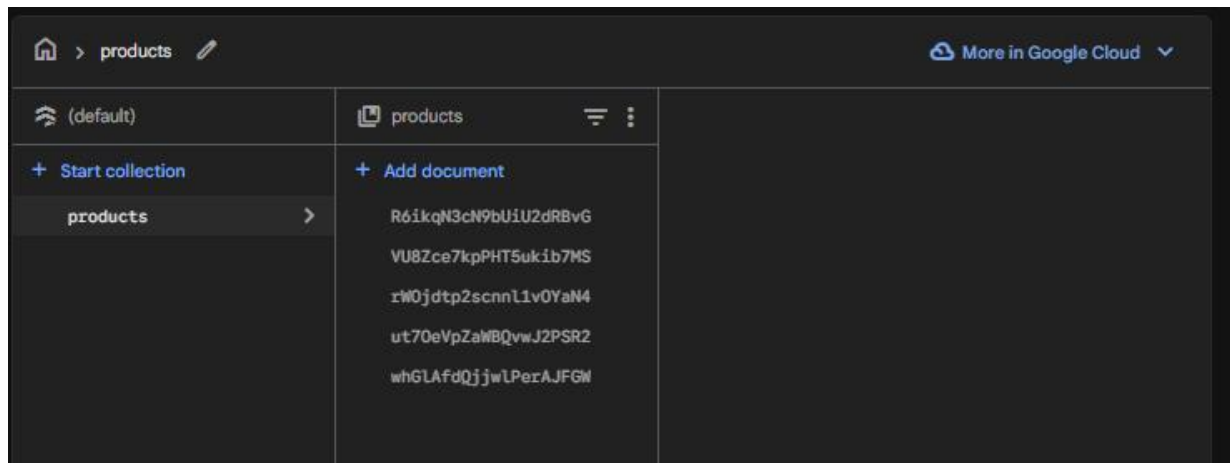
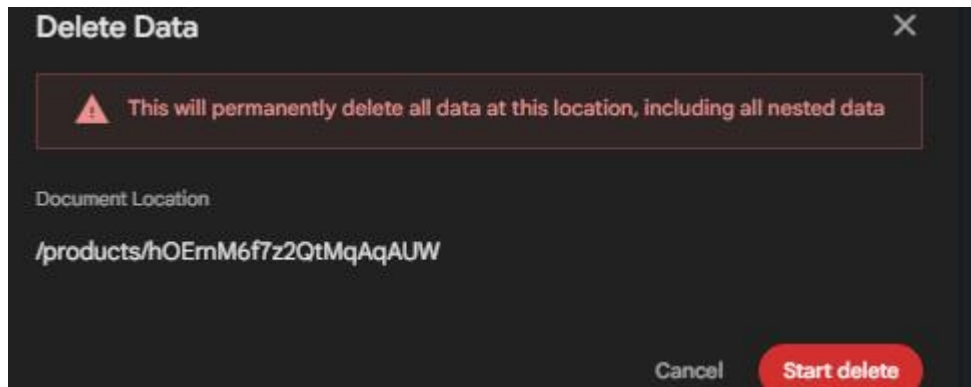
Rp 300,000

celana

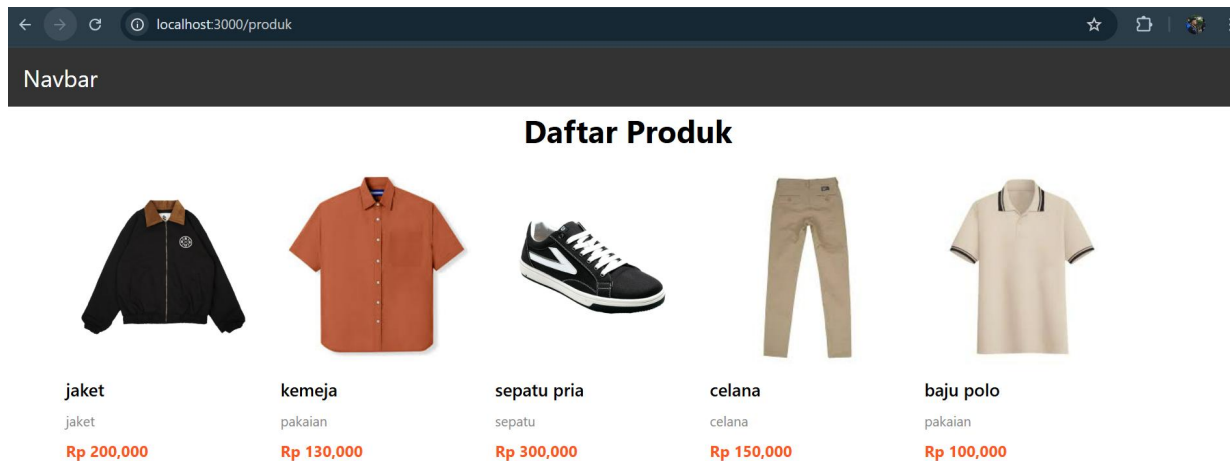
celana

Rp 150,000

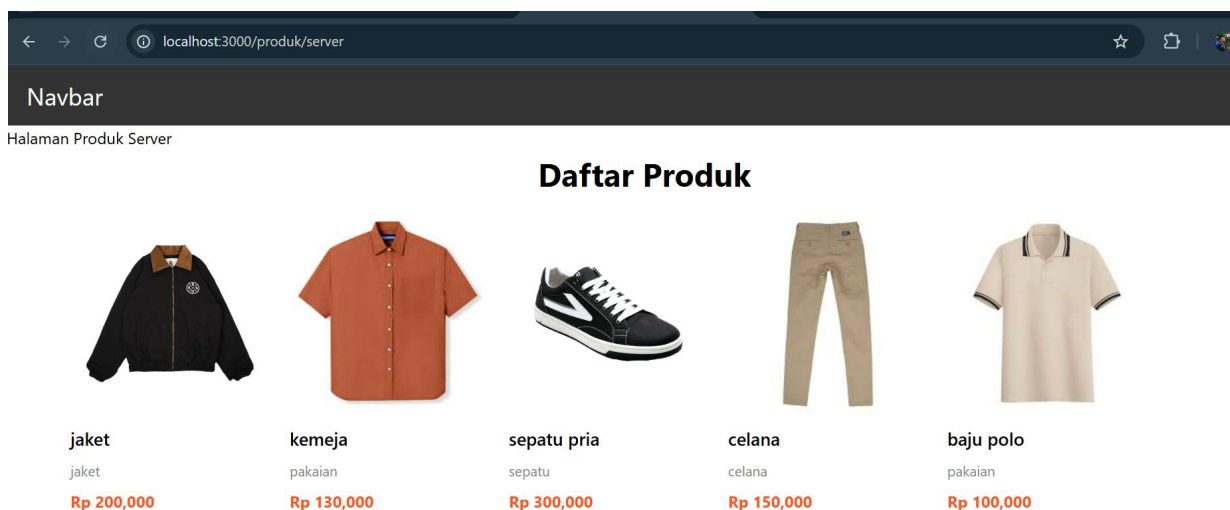
○ Hapus data



/produk (CSR)

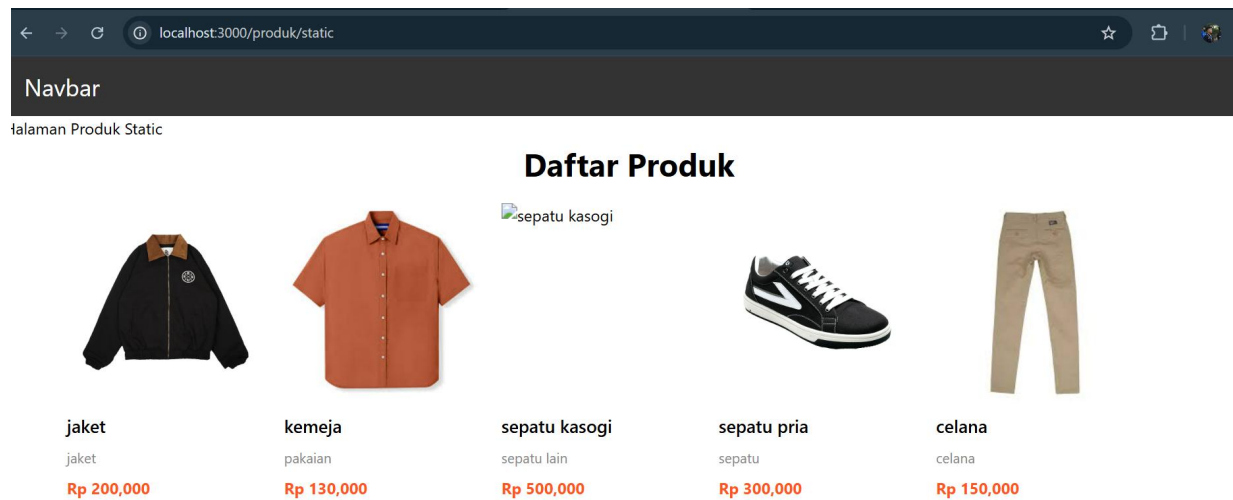


/produk/server (SSR)

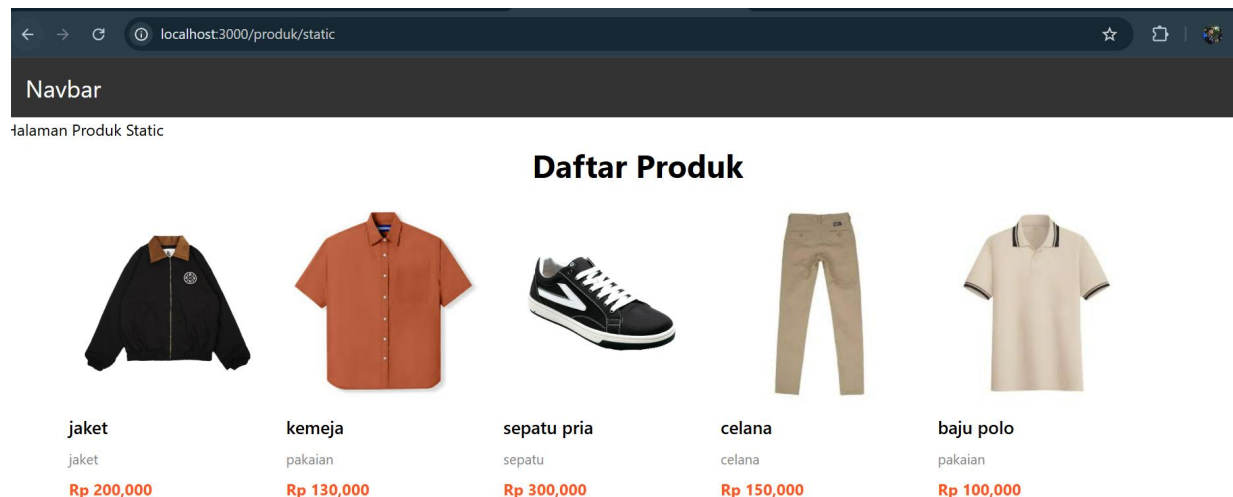


/produk/static (SSG)

-sebelum build ulang



-setelah build ulang



○ Bandingkan hasil

- Client-Side Rendering (CSR) adalah metode di mana data diambil di sisi client atau browser setelah halaman dimuat. Ketika pengguna membuka halaman, server hanya mengirimkan struktur HTML dasar dan file JavaScript. Selanjutnya browser akan mengambil data dari API, kemudian menampilkan konten halaman. Berdasarkan pengujian pada praktikum, ketika data produk di database ditambahkan atau dihapus, halaman /produk (CSR) akan langsung menampilkan perubahan data tanpa perlu melakukan build ulang karena data selalu diambil secara langsung dari API saat halaman dibuka.
- Server-Side Rendering (SSR) adalah metode rendering di mana proses pengambilan data dan pembuatan HTML dilakukan di server setiap kali ada permintaan halaman dari pengguna. Server akan mengambil data terbaru dari database, merender halaman menjadi HTML lengkap, kemudian mengirimkannya ke browser. Berdasarkan hasil pengujian praktikum, halaman /produk/server (SSR) juga langsung menampilkan perubahan data ketika ada penambahan atau penghapusan data di database, karena server selalu mengambil data terbaru setiap kali halaman diakses.
- Static Site Generation (SSG) adalah metode rendering di mana halaman HTML dibuat secara statis saat proses build aplikasi. Data diambil saat proses build menggunakan fungsi seperti `getStaticProps`, kemudian Next.js menghasilkan file HTML statis yang akan dikirim ke browser ketika halaman diakses. Berdasarkan hasil praktikum, ketika data produk di

database ditambahkan atau dihapus, halaman /produk/static (SSG) tidak langsung menampilkan perubahan tersebut. Data baru hanya akan muncul setelah dilakukan proses build ulang aplikasi menggunakan perintah npm run build, karena halaman statis hanya diperbarui saat proses build dilakukan.

E. Studi Analisis

Jawab pertanyaan berikut:

1. Mengapa SSG tidak menampilkan data terbaru?

Jawab : SSG tidak menampilkan data terbaru karena SSG mengambil data saat dilakukan proses build menggunakan “npm run build” bukan pada saat halaman diakses oleh pengguna

2. Mengapa SSG lebih cepat?

Jawab : SSG lebih cepat karena halaman sudah dibuat dalam bentuk HTML statis pada saat proses build. Ketika pengguna mengakses halaman, server hanya perlu mengirimkan file HTML yang sudah tersedia tanpa perlu melakukan proses pengambilan data atau rendering ulang di server.

3. Kapan SSG tidak cocok digunakan?

Jawab : SSG tidak cocok digunakan saat membutuhkan data yang sering berubah ubah

4. Mengapa e-commerce tidak cocok menggunakan SSG murni?

Jawab : Karena data terbaru yang baru ditambahkan tidak akan langsung diakses/dilihat oleh pengguna sebelum adanya proses build ulang

5. Apa perbedaan build mode dan development mode?

Jawab : Development mode adalah mode yang digunakan saat proses pengembangan aplikasi. Pada mode ini server akan terus memantau perubahan kode dan melakukan proses compile ulang secara otomatis sehingga developer dapat langsung melihat perubahan yang dibuat. Sementara itu, build mode adalah proses pembuatan aplikasi dalam bentuk produksi di mana Next.js akan mengoptimalkan kode, mengambil data untuk halaman statis, dan menghasilkan file HTML serta JavaScript yang siap digunakan untuk deployment. Pada build mode inilah halaman SSG dibuat dan data statis diambil untuk pertama kali.